

Implementation and Evaluation of a Multi-Dimensional Resource Defragmentation Policy for the Kubernetes Descheduler

Muhammad Raihan Akbar, and Made Harta Dwijaksana

Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Indonesia

ABSTRAK

Klaster Kubernetes yang dinamis mengakumulasi fragmentasi sumber daya multidimensi yang menimbulkan kapasitas terdampar: CPU dan memori tersedia secara agregat tetapi tersebar dalam bentuk yang tidak dapat dipakai oleh *Pod* baru. Penelitian ini menyajikan *ResourceDefragmentation*, sebuah kebijakan *Balance* pada Kubernetes Descheduler yang bekerja murni berdasarkan *request* sumber daya yang dideklarasikan, tanpa penggunaan aktual. Kebijakan ini mengevaluasi bentuk penggunaan CPU-memori menggunakan *Resource Imbalance Index* (RII), *Free-Space Index* (FSI), dan *bin score*, mengurutkan *Node* sumber, lalu menggusur *Pod* yang *Node* tujuan prediksinya memiliki *bin score* proyeksi tertinggi. *Predictor* target penjadwal memastikan setiap penggusuran memiliki tujuan yang layak, sementara batas utilisasi dan *guard pack-upward* mencegah pengisian berlebih maupun perpindahan ke *Node* yang lebih kosong. Pada lima skenario sumber daya berbasis *request*, kebijakan ini mereklamasikan empat *Node* pada skenario utilisasi rendah, dua pada skenario terfragmentasi, dan satu pada skenario penggabungan *hog*, sementara *HighNodeUtilization* tidak melakukan penggusuran sama sekali pada ketiga skenario tersebut. Pada layout yang *stranding*-nya dapat dikurangi, kebijakan menurunkan skor *stranding* sebesar 73% dan 79%; pada layout yang sudah tak-terkurangi, kebijakan dengan tepat menahan diri dari penggusuran yang tidak produktif. Studi ablasi menunjukkan setiap mekanisme berperan berbeda: pengurutan RII/FSI menjaga kualitas *stranding*, *guard pack-upward* membatasi *churn*, dan pemilih berbasis *bin score* menekan jumlah penggusuran. Setiap eksekusi aman tanpa menyisakan *Pod* berstatus *Pending*. Secara keseluruhan, memilih kandidat penggusuran berdasarkan keseimbangan *Node* tujuan prediksinya sudah cukup untuk mendorong reklamasi *Node* dan defragmentasi dimensional yang efektif.

Kata kunci: Kubernetes, Descheduler, Resource Defragmentation, Bin Packing Multidimensi, Reklamasi Node, Resource Imbalance Index

ABSTRACT

Dynamic Kubernetes clusters accumulate multi-dimensional resource fragmentation that strands capacity: CPU and memory remain available in aggregate but scatter into shapes that no single Node can offer to a new Pod. This study presents *ResourceDefragmentation*, a Kubernetes Descheduler *Balance* policy that operates purely on declared resource requests, without runtime (actual) usage. The policy evaluates CPU-memory shape using the Resource Imbalance Index (RII), Free-Space Index (FSI), and a density-and-balance bin score, orders source Nodes, and evicts the Pod whose predicted scheduler target has the highest projected bin score. A predicted-target check guarantees every eviction has a feasible destination, while a utilization ceiling and a pack-upward guard prevent overfilling a Node or relocating a workload

Universitas Indonesia

to an emptier one. Across five request-based resource scenarios, the policy reclaimed four Nodes in the underutilized scenario and three Nodes in the fragmented scenario, and one in the hog-merge scenario, whereas *HighNodeUtilization* performed no eviction at all in these three. Where stranding was reducible it cut the cluster stranding score by 73% and 79%, and where stranding was already irreducible it correctly declined unproductive evictions. An ablation shows that each mechanism plays a distinct role: RII/FSI ordering preserves stranding quality, the pack-upward guard bounds eviction churn, and the projected-bin-score selector lowers the eviction count. Every run was safe, leaving no Pod Pending. Overall, selecting eviction candidates by the balance of their predicted landing Nodes is sufficient to drive effective Node reclamation and dimensional defragmentation.

Keywords: Kubernetes, Descheduler, Resource Defragmentation, Multi-Dimensional Bin Packing, Node Reclamation, Resource Imbalance Index

INTRODUCTION

Kubernetes schedules Pods by filtering feasible Nodes and scoring the remaining alternatives. This process is effective for initial placement, but running Pods normally remain bound to their Nodes until they terminate or are recreated. As workloads scale and change, free CPU and memory can become scattered into incompatible shapes. Aggregate capacity may be sufficient while no individual Node can accommodate a new Pod, a condition commonly described as resource fragmentation.

Fragmentation affects schedulability, utilization, and infrastructure cost. A Node may retain CPU without enough memory, while another retains memory without enough CPU. These asymmetric residuals strand capacity because the two dimensions cannot be allocated independently (Xiao et al., 2013; Guo et al., 2025). Directed Pod eviction can repair such placement states and reduce the number of Nodes required by the same workload (Larsson et al., 2023).

The Kubernetes Descheduler provides a post-placement correction mechanism by evicting selected Pods and allowing the kube-scheduler to place their replacements (Kubernetes SIG Scheduling, nd). For node reclamation, the important question is not merely whether a source Node is lightly utilized, but whether an evicted Pod has a feasible target that produces denser and better-shaped packing. This requires source-node assessment, target prediction, and an eviction budget that limits disruption. To remain mathematically aligned with the kube-scheduler, a defragmentation policy must evaluate Nodes primarily through their static declared resource requests rather than actual usage, because these static requests dictate placement feasibility and bin-packing shape.

This research develops *ResourceDefragmentation*, a defragmentation-focused Descheduler *Balance* policy that operates purely on declared CPU and memory requests. It identifies drain candidates using utilization and

CPU-memory shape, orders them with a Resource Imbalance Index (RII) and Free-Space Index (FSI), predicts feasible scheduler targets, and chooses the Pod whose predicted landing Node has the strongest projected CPU-memory bin score. The kube-scheduler remains responsible for final binding.

This paper addresses the following research questions:

- RQ1.** How can a Descheduler policy identify and correct CPU-memory fragmentation through directed Pod eviction, and how effectively does it reclaim Nodes and reduce stranded capacity compared to a utilization-based baseline?
- RQ2.** Does predicted-target, bin-score-based eviction-candidate selection improve packing quality and eviction efficiency relative to that baseline?

It makes the following contributions:

- an implementation of *ResourceDefragmentation* as a request-based *Balance* plugin that repairs post-placement scatter without replacing the kube-scheduler;
- a Node-state model combining average utilization, a density-and-balance bin score, RII, and FSI to detect and order fragmented drain candidates; and
- a controlled evaluation across five node-reclamation and fragmentation scenarios, comparing the policy with the upstream *HighNodeUtilization* strategy on node reclamation, dimensional stranding, schedulability headroom, and eviction cost.

LITERATURE REVIEW

This section reviews the theoretical concepts and prior work that form the foundation of this study, including Kubernetes resource management, resource fragmentation, Descheduler mechanisms, and predicted-target eviction-candidate selection.

Kubernetes Architecture and Resource Management

Kubernetes distributes Pods across worker Nodes. For CPU and memory, a container may declare a resource request, a resource limit, both, or neither. Requests represent the resource baseline used by the scheduler when determining whether a Pod fits on a Node, whereas limits are runtime enforcement ceilings applied by the kubelet and container runtime (Kubernetes Authors, ndd). Kubernetes uses effective Pod requests when evaluating placement feasibility and resource allocation on each Node. Resource limits do not participate in this fit calculation; they define runtime enforcement ceilings after the Pod is running. Scheduler scoring strategies such as *MostAllocated* and *RequestedToCapacityRatio* may then use request-based allocation to favor denser packing (Kubernetes Authors, ndc).

Requests and limits also contribute to Pod Quality of Service (QoS) classification. A Pod is *Guaranteed* when every container declares CPU and memory requests equal to its corresponding limits, while Pods with partial or unequal declarations are generally *Burstable*. A Pod whose containers declare neither CPU nor memory requests or limits is *BestEffort* (Kubernetes Authors, ndb). If a limit is declared without a request, Kubernetes can use that limit as the effective request for the resource. A namespace *LimitRange* may also assign default requests or limits during Pod admission. These defaults become part of the Pod's effective resource configuration (Kubernetes Authors, nda).

These decisions are made when a Pod is scheduled. A running Pod is not continuously relocated when the cluster state changes, so an initially acceptable layout can become inefficient after scale-outs, terminations, and replacement events. This separation between initial placement and later cluster state motivates post-placement correction.

Resource Fragmentation and Stranded Capacity

Resource fragmentation occurs when free capacity is divided among Nodes in shapes that do not match incoming workloads. A cluster can have enough aggregate CPU and memory while no individual Node satisfies both dimensions of a Pod request. This makes CPU and memory placement a multi-dimensional problem rather than a scalar load-balancing problem.

CPU-memory skew is one source of stranded capacity. If one Node is CPU-heavy and another is memory-heavy, each leaves a different residual resource that cannot be consumed alone. The absolute difference between normalized CPU and memory fractions provides a direct shape indicator, while the product of free CPU and memory fractions represents the jointly usable free-space region. Prior work shows that directed eviction can repair fragmented Kubernetes layouts and support Node reclamation (Larsson et al., 2023).

Resource Defragmentation and Multi-Resource Bin Packing

Resource packing seeks to place workloads on fewer active Nodes so that empty Nodes can be removed or powered down. In a multi-resource setting, dense packing alone is insufficient: a target can be highly utilized but badly skewed. A useful packing score should therefore represent both density and CPU-memory balance. Container resource defragmentation and task scheduling have been studied using optimization and learning-based methods (Guo et al., 2025; Bagaa et al., 2024), but these are commonly oriented toward placement or scheduling decisions rather than long-term post-placement correction in Kubernetes clusters.

The policy in this research uses normalized CPU and memory fractions. Average utilization represents density, while a bin score combines density with the absolute distance between the two dimensions. The Resource Imbalance Index (RII) preserves the direction

of skew, and the Free-Space Index (FSI) measures the rectangular CPU-memory headroom. Together these quantities identify Nodes that are lightly loaded, poorly shaped, or under pressure from a narrow free-space region.

Kubernetes Descheduler

The Kubernetes Descheduler evicts Pods that violate a policy or contribute to an undesirable placement state, after which normal controllers recreate the Pods and kube-scheduler places them again (Kubernetes SIG Scheduling, nd). It provides an appropriate extension point for node reclamation because it can repair placement without modifying the scheduler.

An eviction policy must nevertheless account for the scheduler boundary. Deleting a Pod does not guarantee that it moves to a useful Node; it may become Pending, return to its origin, or land on a target that worsens fragmentation. A node reclamation-oriented policy therefore benefits from predicting feasible non-origin targets and rejecting movements that exceed a configured ceiling or pack downward.

Upstream utilization policies provide important baselines. *HighNodeUtilization* packs by draining low-utilization Nodes toward more utilized Nodes, but it does not explicitly optimize CPU-memory stranding. This distinction is central to the evaluation in Section 4.

Predicted-Target Eviction-Candidate Selection

Eviction-candidate selection can be formulated around the expected landing state. For every evictable Pod on a drain candidate, *predictSchedulerTarget* simulates request-based placement on non-origin Nodes that satisfy scheduler constraints. Feasible targets must have sufficient CPU and memory, remain below a node reclamation ceiling after placement, and be at least as packed as the source.

The target with the best projected CPU-memory bin score is retained for each Pod. The Pod whose predicted target has the high-

est score is selected for eviction. This single criterion aligns the action directly with the desired destination state: the selected movement should place complementary resource shapes together while preserving node reclamation pressure.

Current Approaches and Research Gap

Existing mechanisms address individual parts of the problem. Scheduler scoring improves new placement, utilization-based descheduling drains Nodes according to thresholds, and fragmentation research motivates directed eviction. The remaining gap is a practical Descheduler policy that combines:

- source assessment using utilization and CPU-memory shape;
- RII/FSI-based drain ordering;
- request-feasible scheduler-target prediction;
- a direct projected-target balance criterion; and
- bounded, convergent eviction.

ResourceDefragmentation is designed to fill this gap while leaving final binding to the standard kube-scheduler.

POLICY DESIGN AND IMPLEMENTATION

This section presents the design and implementation of *ResourceDefragmentation*, a *Balance* plugin for post-placement consolidation in the Kubernetes Descheduler. It builds a snapshot of worker Nodes based on Pod resource requests, finds source Nodes that are either sparse or poorly shaped in CPU-memory space, predicts feasible landing Nodes for their Pods, and evicts the Pod whose predicted scheduler target has the strongest projected CPU-memory bin score. The kube-scheduler remains responsible for final binding.

The experimental configuration explicitly uses declared Pod requests. This keeps Node assessment, feasibility checks, and target prediction aligned with the admission model used by kube-scheduler, and it deliberately excludes runtime (actual) usage so that placement feasibility and bin-packing shape drive every decision.

System Overview

Figure 1 shows the end-to-end control flow of the *ResourceDefragmentation* plugin, from the entry point of the *Descheduler Balance* function to the eviction decision. The plugin operates in two phases: an assessment-and-ordering phase that builds the drain-candidate set, and a bounded eviction loop that relocates Pods from those candidates.

In the first phase, the plugin scans every worker Node. Control-plane Nodes are skipped, and for each remaining Node it computes the average utilization and bin score. A Node whose average utilization and bin score both stay at or above the drain threshold is well-packed and is skipped; otherwise the Node is scored with its Resource Imbalance Index (RII), Free-Space Index (FSI), and priority P_n , added to the drain-candidate set, and the set is sorted in descending priority.

In the second phase, the plugin iterates over the ordered candidates. The loop stops once the per-sweep eviction budget is reached. A Node already marked as a bin, that is, a recent eviction target, is skipped so the same sweep does not drain a Node it has just packed. Otherwise the plugin selects the Pod whose predicted scheduler target has the highest projected bin score; if a feasible target exists the Pod is evicted and the local cache and bin set are updated, and if not the Node is skipped. The remaining subsections formalize each stage: Section 3.2 defines the Node-state metrics, Section 3.3 the drain-candidate detection and ordering, Section 3.4 the predicted scheduler target, and Section 3.5 the eviction-candidate selection.

Node-State Metrics

For Node n , let $u_{n,c}$ and $u_{n,m}$ denote normalized requested CPU and memory:

$$u_{n,c} = \frac{R_{n,c}}{C_{n,c}}, \quad u_{n,m} = \frac{R_{n,m}}{C_{n,m}},$$

where $u_{n,c}$ is the normalized CPU utilization of Node n , $u_{n,m}$ is the normalized memory utilization of Node n , $R_{n,c}$ is the sum of CPU requests from all Pods on Node n , $R_{n,m}$ is the sum of memory requests from all Pods on Node n , $C_{n,c}$ is the allocatable CPU capacity of Node n , and $C_{n,m}$ is the allocatable memory capacity of Node n . Average utilization represents density:

$$U_n^{avg} = \frac{u_{n,c} + u_{n,m}}{2},$$

where U_n^{avg} is the average utilization of Node n , representing overall resource density by taking the mean of normalized CPU and memory utilization.

The bin score is a custom composite scoring function derived from the standard multi-dimensional bin-packing heuristics used in the Kubernetes scheduling plugins (Kubernetes Authors, *ndc*). It fuses two signals that those plugins apply separately, namely the density preference of the *MostAllocated* strategy, which favors well-filled Nodes, and the per-dimension balance preference of the *BalancedAllocation* strategy, which penalizes Nodes whose CPU and memory utilization diverge. Combining density and dimensional balance into a single quantity yields

$$B_n = \frac{\frac{u_{n,c}+u_{n,m}}{2} (1 - |u_{n,c} - u_{n,m}|)}{2},$$

where B_n is the bin score of Node n . The first factor $\frac{u_{n,c}+u_{n,m}}{2}$ is the density term inherited from *MostAllocated*, while the penalty factor $1 - |u_{n,c} - u_{n,m}|$ is the balance term inherited from *BalancedAllocation*, which reduces the score when CPU and mem-

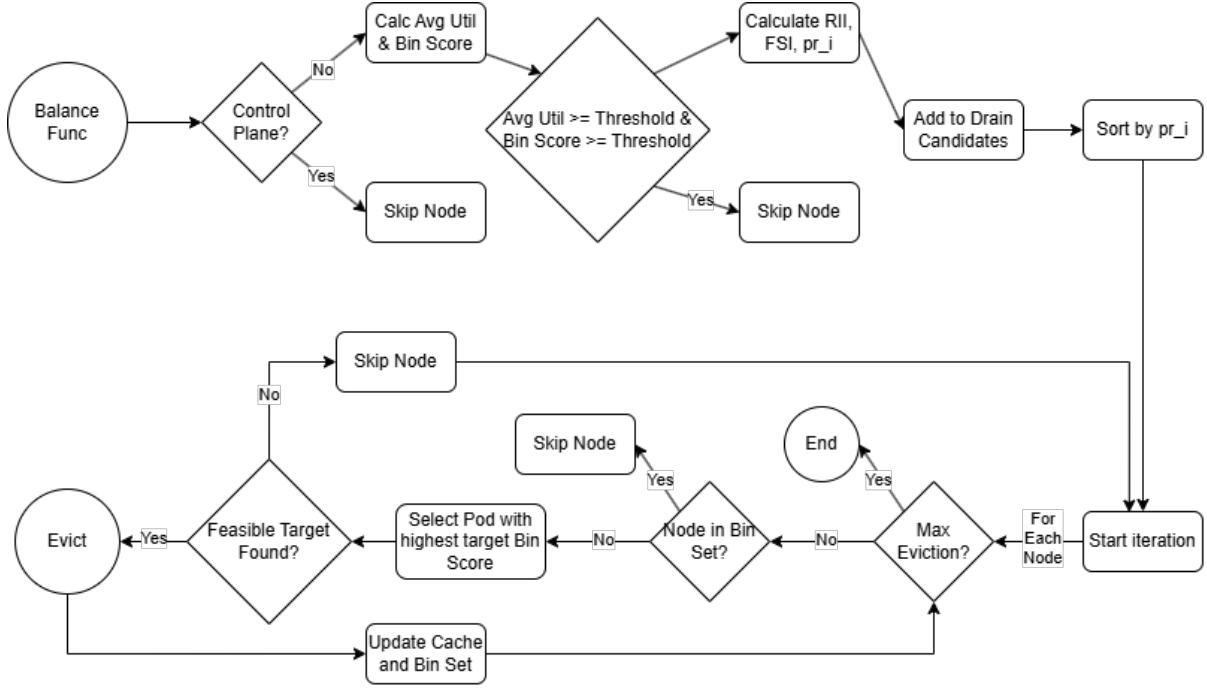


Figure 1. High-level control flow of the *ResourceDefragmentation* plugin, from Node assessment and drain-candidate ordering to the bounded predicted-target eviction loop.

ory utilization diverge. A dense and balanced Node therefore has a high B_n , whereas an empty or strongly skewed Node has a lower score. The signed Resource Imbalance Index, adapted to normalized request fractions from the server imbalance index of (Guo et al., 2025), identifies the skew direction:

$$RII_n = u_{n,c} - u_{n,m},$$

where RII_n is the Resource Imbalance Index of Node n . A positive value indicates CPU-heavy skew (CPU utilization exceeds memory utilization), while a negative value indicates memory-heavy skew. The Free-Space Index, adapted from the free-space size index of (Guo et al., 2025) by expressing the headroom in normalized rather than absolute units, represents joint CPU-memory headroom:

$$FSI_n = (1 - u_{n,c})(1 - u_{n,m}),$$

where FSI_n is the Free-Space Index of Node n , representing the product of remaining

CPU capacity $(1 - u_{n,c})$ and remaining memory capacity $(1 - u_{n,m})$. A higher FSI_n indicates a larger jointly usable headroom region, while a low FSI_n suggests constrained free space in at least one dimension.

Drain-Candidate Detection and Ordering

A worker Node with at least one evictable Pod becomes a drain candidate when either its average utilization or its bin score is below the configured drain threshold τ :

$$U_n^{avg} < \tau \quad \vee \quad B_n < \tau.$$

The disjunction allows the policy to act on two resource-only states. A lightly utilized but balanced Node can be drained to free infrastructure, while a more utilized but badly skewed Node can be drained because its remaining headroom is hard to reuse.

Drain candidates are processed in descending priority:

$$P_n = 0.5 |RII_n| + 0.5 \frac{1}{FSI_n + 1e-10}.$$

The first term emphasizes dimensional skew. The second term emphasizes a narrow jointly usable free-space region. Following the priority formulation of (Guo et al., 2025), the constant is fixed at $1e-10$ and is added to the free-space term of *every* Node as a numerical guard against division by zero, rather than being applied conditionally only when $FSI_n = 0$. Because $1e-10$ is negligible relative to any non-zero $FSI_n \in (0, 1]$, it leaves the relative ordering of drain candidates unchanged for partially loaded Nodes, while bounding the term to a large finite value of $1e+10$ when a Node has no jointly usable headroom ($FSI_n = 0$), so that fully constrained Nodes receive the highest priority. Utilization and bin score decide whether a Node enters the drain set, while RII and FSI decide which drain candidate is handled first.

The two weights of 0.5 generalize the priority formulation of (Guo et al., 2025), which writes the priority as $w_p |RII_n| + (1 - w_p) / (FSI_n + \epsilon)$ for a tunable weight w_p . Setting $w_p = 0.5$ treats dimensional skew and free-space scarcity as co-equal fragmentation signals, so that neither criterion is privileged a priori in the absence of a workload-specific reason to favor one. The two terms nonetheless occupy different numeric ranges: $|RII_n| \in [0, 1]$, whereas $1/(FSI_n + 1e-10) \in [1, 1e+10]$. The free-space term therefore dominates the ordering for densely packed Nodes that have little joint headroom, while $|RII_n|$ acts as a secondary, skew-based tie-breaker among Nodes with comparable free space.

Adjusting this weighting does not change the descheduler as a whole. The priority P_n governs only the *order* in which already-selected drain candidates are processed; it does not decide which Nodes enter the drain set, whether a predicted target is feasible, the pack-upward guard, the utilization

ceiling, or the bounded convergence loop. Re-weighting therefore alters the drain trajectory, namely which Node is handled first and possibly the number of passes required, and can shift the final layout only at the margin, for instance when the pass budget is exhausted before convergence. Because every pass re-evaluates the cluster state and the loop terminates only when a pass produces zero evictions, the policy’s feasibility, safety, and convergence properties are independent of w_p , and the converged layout remains stable across reasonable weightings. The weight is thus a tuning knob for ordering and efficiency rather than a parameter that redefines what the descheduler is permitted to evict.

Predicted Scheduler Target

For every evictable Pod p on source Node s , the plugin examines each non-origin worker Node as a potential relocation target. Let $r_{p,c}$ and $r_{p,m}$ denote the CPU and memory requests of p . Placing p on a target Node t yields the projected utilization

$$u'_{t,c} = \frac{R_{t,c} + r_{p,c}}{C_{t,c}}, \quad u'_{t,m} = \frac{R_{t,m} + r_{p,m}}{C_{t,m}}.$$

A non-origin Node t is admissible only if it is schedulable and p tolerates its taints. An admissible Node is a feasible target when it additionally satisfies the request-fit, utilization-ceiling, and pack-upward conditions, respectively:

$$\begin{aligned} R_{t,c} + r_{p,c} &\leq C_{t,c}, & R_{t,m} + r_{p,m} &\leq C_{t,m}, \\ u'_{t,c} &\leq \theta, & u'_{t,m} &\leq \theta, \\ \min(u'_{t,c}, u'_{t,m}) &\geq \min(u_{s,c}, u_{s,m}). \end{aligned}$$

where θ is the configured target-utilization ceiling, and $u_{s,c}$ and $u_{s,m}$ are the current CPU and memory utilization of source Node s . The request-fit condition ensures that the projected requests do not exceed allocatable capacity in either dimension.

The utilization-ceiling condition caps the projected utilization so that consolidation does not overload a target. The pack-upward guard requires the least-utilized dimension of the target to be at least as loaded as the least-utilized dimension of the source, which prevents an eviction from merely relocating a Pod to an emptier Node.

Let $\mathcal{F}(p)$ denote the set of all feasible targets for p . The function *predictScheduler-Target* returns the feasible Node that maximizes the projected bin score after receiving p :

$$t_p = \arg \max_{t \in \mathcal{F}(p)} B'_t,$$

where B'_t is the bin score B_t evaluated under the projected utilization $(u'_{t,c}, u'_{t,m})$. If $\mathcal{F}(p) = \emptyset$, Pod p has no feasible target and is not evicted.

Predicted-Target Eviction-Candidate Selection

For candidate Pod p , let t_p be its predicted target and B'_{t_p} the target bin score after placement. The selected eviction candidate is:

$$p^\star = \arg \max_p B'_{t_p}.$$

This is the only eviction-candidate selection criterion after target feasibility is checked. It directly prefers a movement that produces the best projected CPU-memory shape at the expected destination. Candidates without a feasible target are excluded. After a successful eviction, the local source and predicted-target snapshots are updated before the next decision. Predicted targets are marked as bins so the same pass does not subsequently drain a Node that has just received packed workload.

Appendix A summarizes the complete execution flow.

Plugin Configuration

EXPERIMENT AND EVALUATION

This section evaluates *ResourceDefragmentation* against the upstream *HighNodeUtilization* strategy across five node-reclamation and fragmentation scenarios, and then ablates the policy's three internal mechanisms to attribute the observed behavior to individual components. All evaluation uses request-space Node state, so placement feasibility and CPU-memory shape, rather than runtime telemetry, drive every comparison.

Experimental Setup

The experiment uses one control-plane Node and six worker Nodes. Each worker provides 2000 *m* allocatable CPU and approximately 811 *Mi* allocatable memory. Pause containers reserve declared requests while contributing negligible runtime load, so in requests mode the declared layout equals the accounted layout and no metrics server is required. Because a packing scheduler would otherwise pack the workloads at deploy time and leave no fragmented starting state, each scenario is seeded deterministically by cordoning all workers and uncordoning them one at a time, then uncordoning all workers before the Descheduler runs.

The scheduler uses *MostAllocated* and *BalancedAllocation*; this packing profile is a precondition, since with the default *LeastAllocated* profile the policy cannot consolidate. Both systems operate on identical initial workloads and namespace scope, system workloads are excluded from eviction, the drain threshold is 0.40, the target-utilization ceiling is 0.90, and the Descheduler is re-run until a pass evicts zero Pods or a five-pass cap is reached. Each policy-scenario cell is a single representative run; metrics are computed by an independent tool that reads only `kubectl get nodes/pods -o json`, so the numbers are comparable across policies and never derived from Descheduler logs.

The two systems under comparison are:

Table 1. ResourceDefragmentation Plugin Arguments

Parameter	Type	Experiment	Purpose
<i>maxEvictions</i>	integer	50	Upper bound on successful eviction attempts in one pass.
<i>consolidationThreshold</i>	float	0.40	Marks a Node as a drain candidate when average utilization or bin score is below this value.
<i>consolidationTarget</i>	float	0.90	Maximum projected CPU or memory utilization on a target Node.
<i>usageMode</i>	string	<i>requests</i>	Pins the experiment to declared requests so all compared policies use the same signal.
<i>namespaces</i>	object	exclude system	Restricts candidate Pods and excludes infrastructure namespaces.

```

1  apiVersion: descheduler/v1alpha2
2  kind: DeschedulerPolicy
3  profiles:
4    - name: default
5      pluginConfig:
6        - name: ResourceDefragmentationC2
7          args:
8            maxEvictions: 50
9            consolidationThreshold: 0.40
10           consolidationTarget: 0.90
11           usageMode: requests
12           namespaces:
13             exclude:
14               - kube-system
15   plugins:
16     balance:
17       enabled:
18         - ResourceDefragmentationC2

```

Listing 1. Request-based ResourceDefragmentation configuration using the ResourceDefragmentationC2 plugin.

- **HighNodeUtilization (HNU):** the upstream utilization-based baseline with CPU and memory thresholds of 40%.
- **ResourceDefragmentation (RD):** the proposed resource-only policy using request-based Node state, RII/FSI-based source ordering, *predictSchedulerTarget*, and projected target bin score as its eviction-candidate criterion.

Scenarios

S1 primarily tests Node reclamation. S2 and S4 test whether dimensional fragmentation can

be reduced when a scalar utilization baseline has no suitable action. S3 combines node reclamation and shape preservation, and S5 stresses eviction-candidate selection during a heterogeneous drain.

Evaluation Metrics

Each run is assessed through four complementary metrics that capture node reclamation, dimensional balance, operational cost, and convergence. Every metric operates on request-space Node state, so the cluster layout is described entirely by per-Node resource requests.

Table 2. ResourceDefragmentation Evaluation Scenarios

Scenario	Intent	Initial condition
S1	Pure node reclamation	Uniformly underutilized Nodes whose workloads can be packed onto fewer workers.
S2	Reducible stranding	Complementary CPU-heavy and memory-heavy Nodes with no clearly high-utilization target.
S3	Mixed objective	Underutilized drain Nodes coexist with already dense balanced Nodes.
S4	Hogs and jumbo Pod	Skewed resource hogs must be merged around a large workload.
S5	Heterogeneous drain	A drain Node contains differently shaped Pods that should land on complementary targets.

The fundamental quantity shared by all metrics is the per-dimension utilization fraction of a worker Node n :

$$u_{n,R} = \frac{R_{\text{req}}(n)}{R_{\text{alloc}}(n)}, \quad R \in \{\text{cpu}, \text{mem}\}$$

where $R_{\text{req}}(n)$ is the sum of resource R requested by all evictable experiment Pods on Node n , and $R_{\text{alloc}}(n)$ is its allocatable capacity for resource R .

Active Node Count The Active Node Count (N_{active}) is the primary indicator of node reclamation. A worker Node is active only when it hosts at least one evictable experiment workload:

$$N_{\text{active}} = \sum_{n=1}^N a_n, \quad a_n = \begin{cases} 1, & \text{if } \sum R_{\text{req}}(n) > 0 \\ 0, & \text{otherwise.} \end{cases}$$

A reduction between the before- and after-snapshots means one or more Nodes were fully drained and became reclaimable for scale-down.

Resource Stranding Score Because defragmentation optimizes the geometric *shape* of utilization rather than its absolute spread, the cluster stranding score S aggregates the ab-

solute CPU-to-memory imbalance across all active Nodes:

$$S = \sum_{n \in N_{\text{active}}} |u_{n,\text{cpu}} - u_{n,\text{mem}}|.$$

A Node with a large gap between its CPU and memory fractions strands the under-used dimension. A lower S indicates that CPU and memory fractions are more tightly aligned across the cluster, unlocking trapped capacity. Unlike a global standard deviation, S is insensitive to intentional differences in absolute load and reacts only to per-Node dimensional skew.

Balanced Headroom The Balanced Headroom (H_{balanced}) converts residual request space into a schedulability-oriented quantity by counting how many reference Pods, each requesting $200m$ CPU and $80Mi$ memory, still fit across the cluster after descheduling. Let $A_{\text{cpu}}(n)$ and $A_{\text{mem}}(n)$ denote the remaining requested CPU and memory space on Node n :

$$H_{\text{balanced}} = \sum_{n=1}^N \min \left(\left\lfloor \frac{A_{\text{cpu}}(n)}{200m} \right\rfloor, \left\lfloor \frac{A_{\text{mem}}(n)}{80Mi} \right\rfloor \right).$$

Because the per-Node term is limited by the tighter dimension, a Node with abundant free CPU but little free memory contributes lit-

the headroom, which is precisely the stranding the policy aims to remove.

Eviction Count and Convergence The Eviction Count (E_{count}) measures the operational cost of a sweep as the total number of Pods relocated, read from the per-pass Descheduler logs. A policy is more efficient when it reaches the same structural improvement with fewer Pod movements, since every eviction disrupts a running workload. Each run is also checked for convergence, that the per-pass count reaches zero within the five-pass cap, and for safety, that no after-snapshot leaves a Pod Pending.

Node Reclamation

ResourceDefragmentation reclaims Nodes precisely in the layouts where HNU performs no eviction, as shown in Figure 2. It reclaims four Nodes in S1, two in S2, and one in S4, whereas HNU is a no-op in all three because it requires Nodes above its threshold to act as packing targets; uniformly low or uniformly fragmented layouts therefore produce no baseline action.

The two cases where HNU reclaims more Nodes than RD, S3 and S5, are exactly the layouts where stranding is already irreducible (Table 4). There RD deliberately declines the extra eviction because it would not improve resource shape; HNU instead empties one more Node greedily. As the ablation in Section 4.7 shows, RD can be made to match that node count by disabling the pack-upward guard, but only at roughly double the eviction cost and with no stranding benefit, so the default trades one reclaimed Node for lower disruption in these irreducible cases.

Stranding Reduction

The stranding results are the core contribution: in the two scenarios where dimensional stranding is reducible, S2 and S4, ResourceDefragmentation cuts it by 73% ($3.495 \rightarrow 0.961$) and 79% ($3.071 \rightarrow 0.633$) by using the predicted-target score to merge complementary CPU-heavy and memory-heavy shapes, while HNU

leaves the layout untouched, as Figure 3 makes visible. This gap, large defragmentation gains exactly where the threshold baseline does nothing, is the central result.

The remaining scenarios delimit the policy honestly. S1 shows a small stranding *cost*: packing six Nodes into two slightly skews the two survivors ($0.045 \rightarrow 0.287$), an acceptable and expected consequence of aggressive consolidation given that the absolute stranding stays negligible while four Nodes are reclaimed. S3 and S5 are already at an irreducible stranding floor, since their complementary shapes cannot be co-packed under the 0.90 ceiling; ResourceDefragmentation correctly holds S constant rather than churning, which is the intended behavior when no shape improvement is feasible. The policy therefore acts where stranding is reducible and stays inert where it is not.

Headroom and Eviction Cost

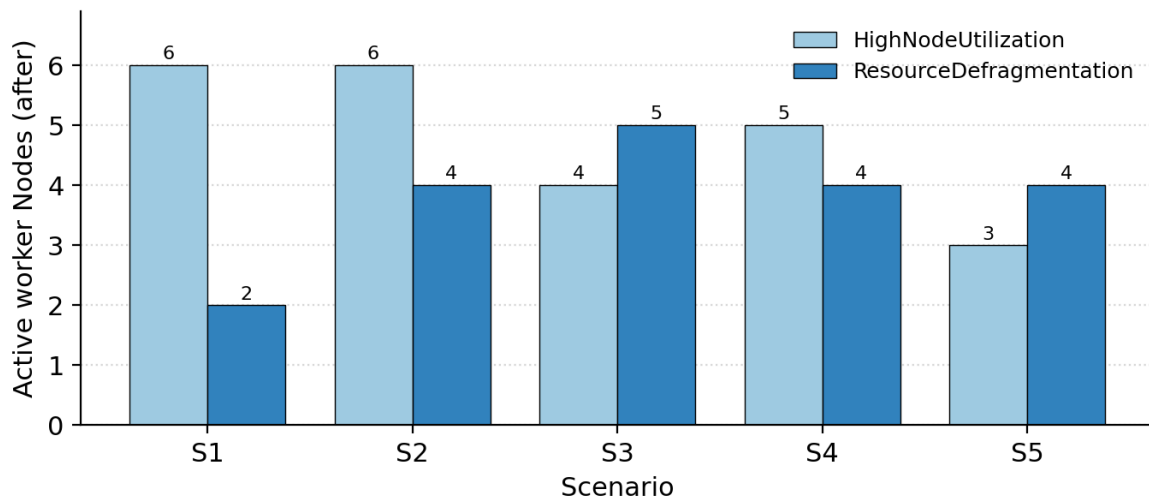
Balanced headroom moves with the resource shape rather than the Node count, as shown in Figures 4 and 5. In the hog-merge scenario S4, ResourceDefragmentation raises balanced headroom from 8 to 13 by rebuilding jointly usable space, the clearest case of defragmentation improving schedulability. In S1–S3 the balanced probe is flat or slightly lower because consolidation concentrates the free capacity onto fewer Nodes; this is expected when Nodes are reclaimed and is captured positively by the reclamation and stranding metrics instead. Where both policies act, S3 and S5, ResourceDefragmentation reaches the same or better cluster state with fewer evictions (S3: 2 vs 4; S5: 1 vs 3), because it stops once no feasible eviction improves the projected shape.

Ablation Study

To attribute the behavior above to individual mechanisms, the full policy is compared against three variants, each disabling exactly one component: **FirstFeasible** replaces the projected-bin-score selection with the first fea-

Table 3. Active Worker Nodes Before and After Descheduling

Scenario	HNU	Reclaimed	RD	Reclaimed
S1	6 $\rightarrow$ 6	0	6 $\rightarrow$ 2	4
S2	6 $\rightarrow$ 6	0	6 $\rightarrow$ 4	2
S3	6 $\rightarrow$ 4	2	6 $\rightarrow$ 5	1
S4	5 $\rightarrow$ 5	0	5 $\rightarrow$ 4	1
S5	4 $\rightarrow$ 3	1	4 $\rightarrow$ 4	0

**Figure 2.** Active worker Nodes after descheduling per scenario.**Table 4.** Cluster Stranding Score Before and After Descheduling

Scenario	HighNodeUtilization	ResourceDefragmentation
S1	0.045 $\rightarrow$ 0.045	0.045 $\rightarrow$ 0.287
S2	3.495 $\rightarrow$ 3.495	3.495 $\rightarrow$ 0.961
S3	1.489 $\rightarrow$ 1.489	1.489 $\rightarrow$ 1.489
S4	3.071 $\rightarrow$ 3.071	3.071 $\rightarrow$ 0.633
S5	1.335 $\rightarrow$ 1.335	1.335 $\rightarrow$ 1.335

Table 5. Balanced Headroom (after) and Eviction Count

Scenario	HNU $H_{balanced}$	HNU E	RD $H_{balanced}$	RD E
S1	18	0	18	8
S2	15	0	12	10
S3	8	4	7	2
S4	8	0	13	6
S5	16	3	16	1

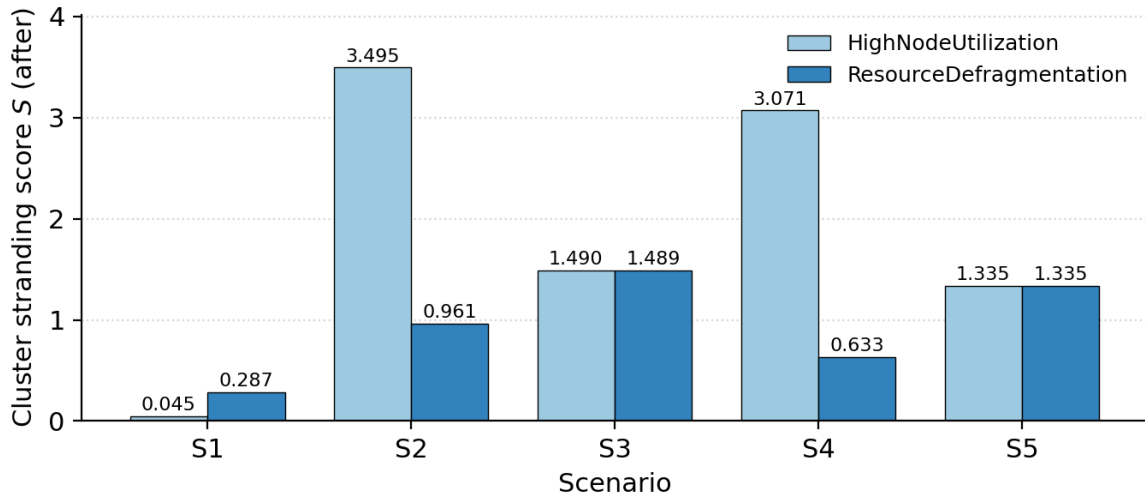


Figure 3. Final cluster stranding score S per scenario after descheduling (lower is better).

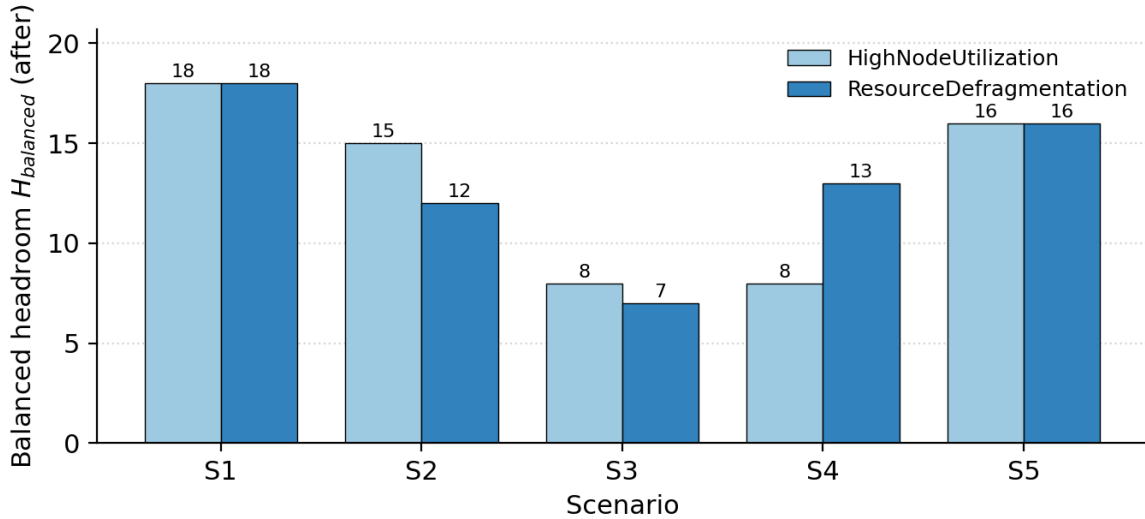


Figure 4. Balanced headroom $H_{balanced}$ after descheduling per scenario.

sible target; **NoPackUpward** removes the pack-upward guard; and **NoOrder** drains candidates in arbitrary order instead of by the RII/FSI priority. Table 6 reports the final stranding S and total eviction count E for each variant.

As Figure 6 shows, each mechanism earns its place, but on a different axis:

- **RII/FSI ordering protects stranding quality.** Disabling it regresses S4 sharply, from 0.633 to 1.581, because handling drain candidates in the wrong order relocates Pods before

the complementary target is available. Elsewhere the ordering does not change the converged S , so its role is to preserve defragmentation quality in the shape-sensitive case.

- **Pack-upward controls churn.** NoPackUpward reaches the same stranding as Full but at a higher eviction cost (S1: 12 vs 8; S4: 11 vs 6). Without the guard it also empties an extra Node in S3 ($6 \rightarrow 4$ vs $6 \rightarrow 5$) and S5 ($4 \rightarrow 3$ vs $4 \rightarrow 4$), but that node-count gain costs roughly

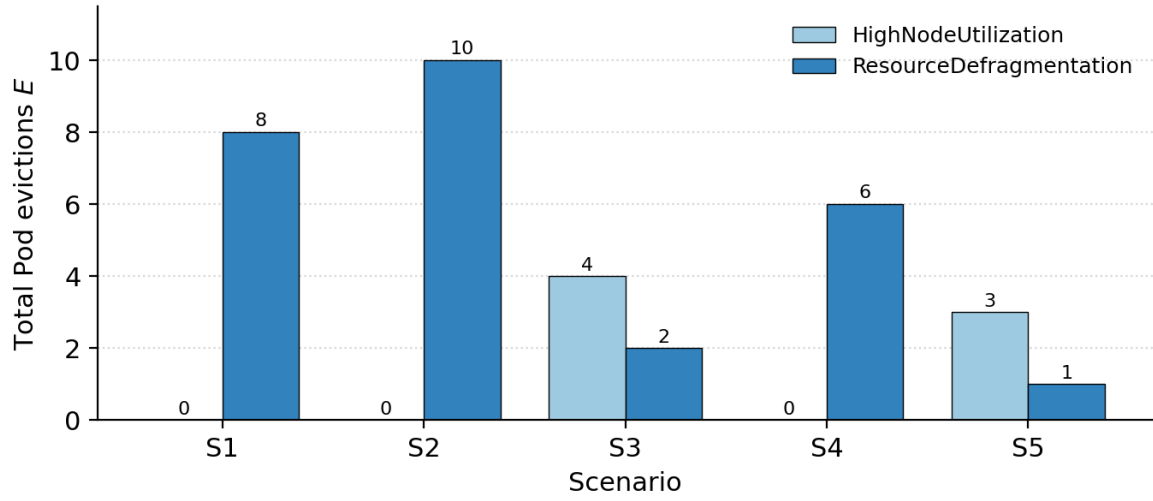


Figure 5. Total Pod evictions E per scenario.

Table 6. Ablation: Final Stranding S (Eviction Count E) per Variant

Scenario	Full	FirstFeasible	NoPackUpward	NoOrder
S1	0.287 (8)	0.287 (12)	0.287 (12)	0.287 (10)
S2	0.961 (10)	0.961 (10)	0.961 (10)	0.961 (10)
S3	1.489 (2)	1.489 (2)	1.489 (4)	1.489 (2)
S4	0.633 (6)	0.633 (6)	0.596 (11)	1.581 (4)
S5	1.335 (1)	1.335 (1)	1.335 (3)	1.335 (1)

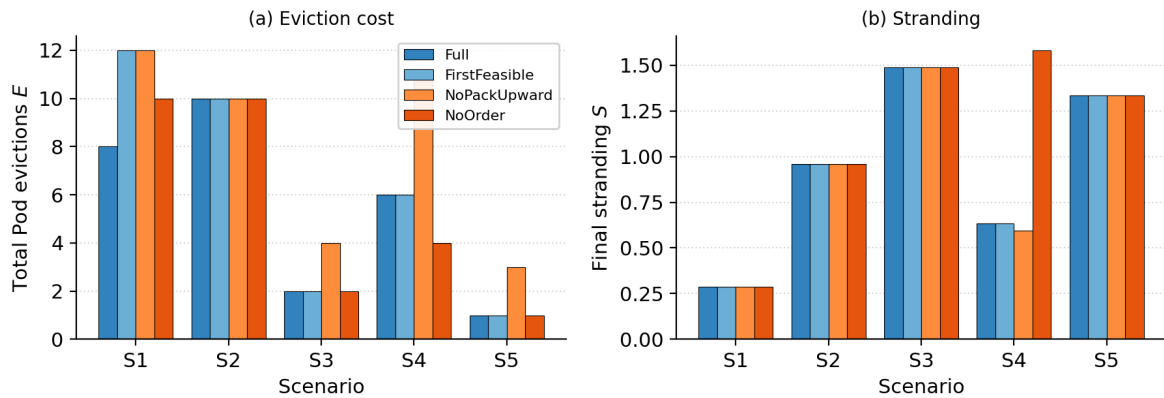


Figure 6. Ablation across the four variants: (a) eviction cost E and (b) final stranding S per scenario. Disabling pack-upward inflates eviction cost (S4), and disabling the RII/FSI ordering regresses S4 stranding, while the bin-score selector chiefly lowers eviction count.

double the evictions and yields no stranding improvement, so it is a disruption regression rather than a win.

- **The bin-score selector is low-churn selection.** FirstFeasible matches Full on stranding in every scenario, so the projected-bin-score criterion does not drive stranding on this cluster. Its measurable effect is efficiency: Full reaches the same S1 result with 8 evictions where FirstFeasible needs 12.

The correct reading is therefore that the stranding wins come from the consolidation-and-defragmentation pipeline as a whole, candidacy, ordering, ceiling, and pack-upward, beating the utilization-threshold baseline, while the predicted-target selector keeps the eviction count down. Because each component either protects stranding quality or reduces churn, the default keeps all three enabled; a documented *disablePackUpward* knob can expose the higher-reclamation, higher-churn mode for operators who explicitly want maximum node reclamation.

Convergence and Safety

Four of the five scenarios reach a zero-eviction pass within two or three passes without oscillation. S2 is the exception: it continues to make one single-Pod move per pass and reaches the five-pass cap without a zero pass, because the fragmented layout admits a long tail of marginally improving relocations after the initial bulk consolidation. In every scenario, including S2, the run remains safe: no after-snapshot contains a Pending Pod, all evicted Pods are controller-owned and eligible under the default evictor, infrastructure namespaces are excluded, and no eviction request is left outstanding. These observations validate the feasibility guard, target ceiling, and bounded execution loop, and indicate that the pass cap, rather than an oscillation, is what bounds the tail case.

Resource Policy Findings

The evaluation yields four main findings:

- ResourceDefragmentation reduces dimensional stranding by 73% in S2 and 79% in S4 and reclaims Nodes in S1, S2, and S4, all layouts in which HNU performs no eviction at all.
- On layouts where stranding is already irreducible (S3, S5), the policy correctly declines unproductive evictions, trading at most one reclaimed Node for substantially lower churn.
- The ablation attributes each mechanism to a distinct role: RII/FSI ordering preserves stranding quality (S4), pack-upward bounds eviction churn, and the projected-bin-score selector lowers eviction count without changing stranding.
- Every run is safe, leaving no Pod Pending; four of five scenarios converge within three passes, with S2 bounded by the five-pass cap rather than oscillation.

Threats to Validity

The evaluation is designed to isolate the request-based defragmentation mechanism, and this focus introduces limitations that bound how far the results should be generalized.

Construct validity. The experiment describes each Node purely through the declared CPU and memory requests of its Pods, and workloads are pause containers that reserve those requests while producing negligible runtime load. This is a deliberate consequence of the request-based scope of *ResourceDefragmentation*: because the kube-scheduler admits Pods on effective requests, request-space state is the correct signal for placement feasibility and bin-packing shape. It does mean, however, that the reported reclamation and stranding metrics reflect the declared-request layout rather than live consumption. Divergence between

Table 7. Per-Pass Eviction Counts

Scenario	HighNodeUtilization	ResourceDefragmentation
S1	0	6, 2, 0
S2	0	6, 1, 1, 1, 1
S3	4, 0	2, 0
S4	0	2, 2, 1, 1, 0
S5	3, 0	1, 0

requests and actual usage, and the service-level cost of evicting a busy Pod, are outside the scope of this policy and are addressed by a separate actual-usage pre-eviction guard rather than measured here.

Internal validity. Each policy-scenario cell is a single representative run rather than a multi-seed average, so small per-run variation is not quantified; every run is nonetheless checked for convergence and for the absence of Pending Pods. The ablation in Section 4.7 isolates the contribution of each component by disabling one mechanism at a time under identical conditions, which strengthens the attribution of the observed effects. The comparison against external systems still uses a single baseline, *HighNodeUtilization*, because it is the upstream, in-tree consolidation strategy and therefore a faithful production reference; reimplementing prior proposals that publish only pseudocode, without released code or artifacts, would risk an unfaithful comparison whose behavior reflects our reconstruction rather than the original design.

External validity. All experiments run on a single small cluster of one control-plane Node and up to six homogeneous AWS EC2 worker Nodes, each with an identical CPU and memory profile. The five scenarios are hand-constructed to exercise specific fragmentation shapes rather than sampled from production traces, so they demonstrate the mechanism under controlled conditions but are not a representative distribution of real cluster layouts. Larger clusters, heterogeneous Node capacities, multi-tenant workloads, and additional

resource dimensions such as storage and GPU may change both the magnitude of the gains and the eviction cost, and evaluating the policy at scale is left to future work.

Scheduler alignment. *ResourceDefragmentation* decides which Pods to evict, but final binding remains the responsibility of the kube-scheduler, and the evaluation confirmed that the intended consolidation is realized only under the *MostAllocated* and *BalancedAllocation* packing profile. The predicted target is an estimate of where a replacement will land; if the active scheduler profile is not aligned with the policy’s intent, an evicted Pod may be placed on a Node other than the predicted target, which can reduce the realized defragmentation quality.

CONCLUSION AND FUTURE WORK

This research designed and evaluated *ResourceDefragmentation*, a request-based Kubernetes Descheduler *Balance* policy that repairs post-placement CPU-memory fragmentation without replacing the kube-scheduler.

Identifying fragmented Nodes. *ResourceDefragmentation* identifies fragmented Nodes directly from their request-space resource shape, without relying on runtime usage or a single utilization threshold. It selects drain candidates from average utilization and CPU-memory bin quality, orders them by the Resource Imbalance Index (RII) and Free-Space Index (FSI), and then evicts the Pod whose predicted landing Node, found through *predictSchedulerTarget*, has the best projected bin score. A target ceiling and pack-upward guard

prevent overfilling a Node or relocating a workload to an emptier one. Together the drain ordering, predicted-target criterion, and guards drive effective packing while keeping eviction churn low.

Effectiveness against the baseline. ResourceDefragmentation reclaimed four Nodes in S1, two in S2, and one in S4, all cases where *HighNodeUtilization* performed no eviction. It cut dimensional stranding by 73% in S2 ($3.495 \rightarrow 0.961$) and 79% in S4 ($3.071 \rightarrow 0.633$) in the layouts where stranding is reducible, and on the already-irreducible layouts (S3, S5) it correctly declined unproductive evictions rather than churning. Four of five scenarios converged within three passes and no run left a Pod Pending.

Component attribution. An ablation that disabled one mechanism at a time showed that each earns its place on a different axis: RII/FSI ordering preserves stranding quality (removing it regressed S4 from 0.633 to 1.581), the pack-upward guard bounds eviction churn (removing it roughly doubled evictions for no stranding gain), and the projected-bin-score selector lowers the eviction count without changing the converged stranding. The stranding wins therefore come from the consolidation-and-defragmentation pipeline as a whole, while the selector keeps disruption low.

Overall, the results support a simple conclusion: selecting eviction candidates by the balance of their predicted landing Nodes, combined with request feasibility, bounded utilization, drain ordering, and pack-upward behavior, is enough to drive useful Node reclamation and dimensional defragmentation where it is achievable while avoiding wasteful eviction where it is not.

Future Work

- **Broader Resource Dimensions:** Extend the framework to additional resource dimensions, including storage and GPU, so that target prediction and bin-quality scoring can handle

workloads beyond CPU and memory.

- **Scheduler Coordination:** Establish and evaluate scheduler coordination mechanisms that expose the predicted target, or a compatible scoring signal, to the kube-scheduler, ensuring that predicted-target placements align with actual scheduling decisions and eliminating the misalignment that currently bounds maximum defragmentation precision.
- **Pending-Pod Awareness:** Introduce Pending-pod awareness into the eviction pipeline so that drain candidates are selected with knowledge of already-unschedulable workloads, ensuring evictions free the right resource type on the right Node to unblock Pending Pods rather than delaying them further.
- **Threshold Sensitivity Analysis:** Conduct a sensitivity analysis of the drain threshold, target ceiling, and related parameters across various workloads and cluster types to characterize their effect on node reclamation, stranding reduction, and convergence stability.
- **Broader Generalizability Evaluation:** Evaluate the plugin across broader cluster shapes, node heterogeneity, and workload mixes to assess generalizability beyond the five scenarios studied.
- **Broader Baselines and Multi-Seed Runs:** Extend the comparison beyond *HighNodeUtilization* once faithful reference implementations of prior pseudocode-only proposals become available, and repeat each policy-scenario cell across multiple seeds to quantify run-to-run variation.

REFERENCES

Bagaa, M. et al. (2024). A multi-objective framework for power-aware scheduling in kubernetes. *Depot-e UQTR*.

Universitas Indonesia

Guo, Y., Wei, L., Fan, C., Ma, Y., Zhao, X., and He, H. (2025). Joint optimization of container resource defragmentation and task scheduling in queueing cloud computing: A drl-based approach. *Future Internet*, 17(11):483.

Kubernetes Authors (n.d.a). *Limit Ranges*. <https://kubernetes.io/docs/concepts/policy/limit-range/>.

Kubernetes Authors (n.d.b). *Pod Quality of Service Classes*. <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>.

Kubernetes Authors (n.d.c). *Resource Bin Packing*. <https://kubernetes.io/docs/concepts/scheduling-eviction/resource-bin-packing/>.

Kubernetes Authors (n.d.d). *Resource Management for Pods and Containers*. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>.

Kubernetes SIG Scheduling (n.d.). *Descheduler for Kubernetes*. <https://github.com/kubernetes-sigs/descheduler>.

Larsson, O., Klein, C., and Elmroth, E. (2023). The impact of directed pod eviction on kubernetes resource utilization. In *2023 IEEE 17th International Conference on Service-Oriented System Engineering (SOSE)*, pages 78–87.

Xiao, Z., Song, W., and Chen, Q. (2013). Dynamic resource allocation using virtual machines for cloud computing environment. *IEEE Transactions on Parallel and Distributed Systems*, 24(6):1107–1117.

RESOURCE DEFRAGMENTATION BALANCE FILTER ALGORITHM

```

1  Function: ResourceDefragmentationBalance
2  Input:
3      nodes
4      maxEvictions
5      drainThreshold
6      targetUtilization
7  Output:
8      evictedPods
9
10     states <- BuildRequestBasedNodeStates(nodes)
11     drainCandidates <- empty list
12
13     for each worker node n in nodes do
14         avg <- AverageUtilization(states[n])
15         bin <- BinScore(states[n])
16         rii <- ResourceImbalanceIndex(states[n])
17         fsi <- FreeSpaceIndex(states[n])
18         pods <- GetEvictablePods(n)
19
20         if pods is not empty and
21             (avg < drainThreshold or
22              bin < drainThreshold) then
23             priority <- 0.5 * abs(rii) +
24                       0.5 * (1 / (fsi + epsilon))
25             Add(drainCandidates, {n, pods, priority})
26         end if
27     end for
28
29     SortDescending(drainCandidates, by priority)
30     bins <- empty set
31     evictionCount <- 0
32
33     for each drain in drainCandidates do
34         if evictionCount >= maxEvictions then break
35         if drain.node in bins then continue
36
37         remaining <- drain.pods
38         while remaining is not empty and
39             evictionCount < maxEvictions do
40             bestPod <- empty
41             bestTarget <- empty
42             bestScore <- negative infinity
43
44             for each pod p in remaining do
45                 target <- PredictFeasibleTarget(
46                     p,
47                     drain.node,
48                     states,
49                     targetUtilization,
50                     requirePackUpward = true)
51
52                 if target exists then
53                     projected <- StateAfterPlacement(states[target], p)
54                     score <- BinScore(projected)
55                     if score > bestScore then
56                         bestPod <- p
57                         bestTarget <- target
58                         bestScore <- score
59                     end if
60                 end if
61             end for
62
63             if bestPod is empty then break
64
65             if Evict(bestPod) succeeds then
66                 evictionCount <- evictionCount + 1
67                 UpdateSourceAndTargetStates(
68                     states, drain.node, bestTarget, bestPod)

```

```
69         Add(bins, bestTarget)
70     end if
71     Remove(remaining, bestPod)
72 end while
73 end for
74
75 return evictedPods
```

Listing 2. ResourceDefragmentation predicted-target eviction workflow.